Impact of Similarity Measures in Graph-Based Automatic Text Summarization of Konkani Texts

JOVI D'SILVA*

Assam Don Bosco University, Guwahati, Assam, India, jovidsilva@gmail.com

UZZAL SHARMA

Assam Don Bosco University, Guwahati, Assam, India, uzzal.sharma@dbuniversity.ac.in

Automatic text summarization is a popular area in Natural Language Processing and Machine Learning. In this work, we adopt a graph-based text summarization approach, using PageRank algorithm, for automatically summarizing Konkani text documents. Konkani is an Indo-Aryan language spoken primarily in the state of Goa, which is on the west coast of India. It is a low-resource language with limited language processing tools. Such tools are readily available in other popular languages of choice for automatic text summarization, like English. The Konkani language dataset used for this purpose is based on Konkani folktales. We examine the impact of various language independent and language dependent similarity measures on the construction of the graph. The language dependent similarity measures use pre-trained fastText word embeddings. A fully connected undirected graph is constructed for each document with the sentences represented as the graph's vertices. The vertices are connected to each other based on how strongly they are related to one another. Thereafter, PageRank algorithm is used for ranking the scores of the vertices. The top ranking sentences are used to generate the summary. ROUGE toolkit was used for evaluating the quality of these system generated summaries and the performance was evaluated against human generated 'gold-standard' abstracts, and also compared with baselines and benchmark systems. The experimental results show that language independent similarity measures performed well compared to language dependent similarity measures despite not using language specific tools, like stop-words list, stemming and word embeddings.

CCS CONCEPTS • Information retrieval → Retrieval tasks and goals → Summarization

Additional Keywords and Phrases: TextRank, Automatic Text Summarization, PageRank, Graph-based method, fastText, Similarity measures, Konkani, Low-resource language

1 INTRODUCTION

Text summarization involves creating a summary that outlines the key points of a lengthy text document [1, 2]. A reader could get the gist of the contents of a document by just reading this summary. Enormous volumes of text documents are frequently uploaded on the Internet, and summarizing these humongous amounts of textual data is a rather laborious task if it were to be accomplished by humans. However, with the advancements in Machine Learning (ML), this task of summarization can be automated with better efficacy. This process is precisely termed as 'Automatic Text Summarization' (ATS). The summaries generated by an ATS system can be broadly divided into two categories viz. abstractive summaries and extractive summaries [1, 2, 3]. An abstractive summary is generated by interpreting and rewording the contents of a text document. An extractive summary is generated by compiling certain 'key sentences' picked up from the original document. These 'key sentences' selected are such that they accurately illustrate the essence of the document [4, 5]. Machine learning approaches for text summarization can be classified as language dependent approaches and language independent approaches [6]. Language dependent approaches rely on language specific tools such as a lexicon of words for stop-words elimination, lemmatizers, pre-trained word

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Association for Computing Machinery.
2375-4699/2022/1-ART1 $15.00
https://doi.org/10.1145/3554943

Just Accepted

embeddings and so on for the generation of summaries. Whereas language independent approaches are the ones that do not use any such tools for summary generation.

Advancements and research in Natural Language Processing (NLP) have paved way towards language independent text summarization methodologies that can assist in summarization task irrespective of the language of the source document [7, 8]. Graph-based techniques are popular amongst the language independent approaches [8]. The human-generated summaries are mostly abstractive [10], whereas summaries produced by automatic summarization tools are mainly extractive in nature. Natural languages used by humans are not natively understood by computer systems and therefore, a computer can find it difficult to produce an abstractive summary with the same quality standard as a human-generated summary. However, producing an extractive summary can be relatively simpler [10].

Graph-based approach was originally proposed as a language independent technique and these techniques have also been proven to be the best in Document Understanding Conference (DUC) experiments [11]. In graph-based approach, an undirected graph is constructed based on a source document, with the nodes representing the sentences. The edges in the graph connect two nodes (or sentences). The weights assigned to the edges determine how closely they are related to each other. This relationship among the sentences is usually defined by similarity measures, which are evaluated during the graph formation step and assigned to the edges as weights. This is followed by applying the "PageRank" algorithm to the graph for generating a ranking of the sentences. The top ranked sentences are then used to form the summary [12]. In this paper, we examine language independent similarity measures and also language dependent similarity measures which are based on pre-trained word embeddings, for performing graph-based extractive summarization. We demonstrate the use of graph-based ranking algorithm, PageRank, for extractive summarization of Konkani text documents and explore the impact of similarity measures on graph formation.

Konkani is an Indo-Aryan language spoken primarily by the people of Goa. It is the official language of the state of Goa, located on the west coast of India and is written in Devanagari script. It is also spoken by some minority groups along the 'Konkan' region of western India. The number of Konkani speakers is about 2.2 million, which is ~0.19% of Indian population, according to 2011 census [13]. The percentage of people speaking the language has dropped since the census conducted in 2001. We devised a special dataset in Konkani language to explore ATS in Konkani [3]. The dataset is based on books on Konkani folktales and is written in Devanagari script.

Most of the research pertaining to ATS worldwide encompasses popular languages like English, Arabic etc. Indian languages like Hindi, Malayalam, Bengali, Marathi have also been favoured languages by most researchers in India. However, there have so far not been many attempts in the domain of ATS for this low-resource regional language, Konkani. A low-resource language is the one with limited datasets and/or pre-existing language processing tools, such as lemmatizers, stop-words list etc. The lack of availability of such tools poses as an added challenge for performing text summarization. Therefore, a language independent approach would be beneficial for performing text summarization in such a language. And, such a system could also be extended to other low-resource languages. However, Facebook has made available pre-trained word embeddings which have shown tremendous growth and importance in NLP research in recent years. These have also been examined to compute similarity measures, but word embeddings are language dependent. One of the key drivers for the development of text summarization systems is to help sustain the digital growth of Konkani language by providing the readers with summarized digital texts on Konkani literature and allowing users to consume this summarized digital content.

Just Accepted

The major contributions of this study are:

1. This work is a part of one of the first attempts to study graph-based methods used to perform extractive automatic text summarization of Konkani language folktales extracted from books.
2. To examine the use of language independent similarity measures and language dependant similarity measures on graph formation and the resulting summaries.
3. To evaluate the effectiveness of unsupervised graph-based methods for text summarization on Konkani, a low-resource language.
4. To evaluate the summaries generated by the system using ROUGE Toolkit with Lead Baseline, Modified Luhn's Heuristic Baseline and Human-annotated Benchmark.

The paper layout is as follows, Section 2 highlights the previous work completed using graph-based approach and also the studies conducted that used word embeddings along with graph-based approach for performing ATS. Section 3 gives the step-by-step methodology used in this study to automatically generate summaries. Section 4 gives the details of the Konkani language dataset specially compiled for ATS study in Konkani. The details of the language dependent and the language independent similarity measures are presented in Section 5. The results and evaluations of the results are given in Section 6. The last section, Section 7, presents the conclusion of this study.

2 PREVIOUS WORK

We give an overview of the work carried out in ATS using graph-based language independent approach and thereafter; we review some initiatives that use a language dependent approach using word embeddings with graph-based technique for performing ATS. Graph-based methods are unsupervised ATS approaches that use of graph theoretic concepts. Being unsupervised, these approaches eliminate the need for any training data. Mihalcea and Tarau first introduced a graph-based ranking system, "TextRank", for text processing in 2004 [14]. It also illustrates two unsupervised approaches for sentence and keyword extraction. "TextRank" algorithm is based on "PageRank" algorithm used by Google for ranking web pages [12]. Another graph-based method, "LexRank" was proposed in 2004, which determines relevance of sentences based on "eigenvector centrality" in a graph-based sentence representation [15]. Graph-based methods have gathered the attention of a lot of researchers in the ATS domain recently. These methods have also provided promising output [16, 17, 18] and have proven to be efficient in various text-based ranking applications.

HITS and PageRank are graph ranking algorithms [14,18]. PageRank is the most popular algorithm among the graph-based ranking algorithms for automatically extracting sentences [14]. A study comparing the two ranking algorithms, HITS and PageRank, found that PageRank's performance exceeded that of HITS when executed on the same Konkani dataset [19].

In 2009, Lin et al. proposed an unsupervised approach for creating extractive summaries by constructing a semantic graph on the original source documents [20]. The results of the proposed system exhibited promising outcome and the performance could be further improved by building an enhanced graph with rich features. Ferreira et al. illustrated a four-dimensional graph model that considers the following factors viz. Co-reference, similarity, discourse information and semantic similarity, for generating a graph [16]. Their model outperformed the current systems qualitatively and quantitatively. In 2018, Mallick et al. implemented a modified version of "TextRank" approach and proposed the use of a modified sentence scoring based on "inverse sentence frequency-modified-cosine" similarity measure [21]. Their

Just Accepted

approach was effective for extractive summarization; however, there were possibilities that multiple similar sentences with high score were chosen to be included in the summary.

Most of the research pertaining to ATS has been done on the most commonly spoken languages of the world. Various avenues have been widely explored in English language pertaining to the automatic text summarization domain. The Indian-subcontinent houses a wide range of linguistic diversity. Some research has been done on languages like Hindi, Bengali, Tamil, and Marathi etc. Banu and Karthika proposed a technique for Automatic summarization of Tamil documents by using rule-based approach to construct a semantic-graph representation of source documents. A statistical methodology is then implemented to build sub-graphs that reflect the relevant portions of the source document, which can be considered to be a part of the document summary [22]. Sankar et al. proposed a graph-based text extraction method for automatic text summarization in Tamil that ranks the sentences based on two measures: the first being "Word Frequency Analysis" and the second being "weight calculation" depending on word position and string pattern [23]. Their findings show that the unsupervised approach works better in a time-efficient manner and does a decent job with no training corpus.

Kumar et al. proposed an approach for extractive text summarization for Hindi language using graph-based approach. This technique used sentence ranking algorithm to mark relevant sentences, and thereafter, employed semantic analyses to determine relations between sentences [24]. The system achieved improved results in terms of precision, recall and F-measure. Dalal and Malik implemented a 'Semantic Graph Model' for the automatic summarization of Hindi text [25]. They constructed a semantic graph by extracting "subject-object-verb" triples from the source documents. They have then used "Particle Swarm Optimization (PSO)" algorithm to train a classifier, which produces a semantic sub-graph and generates a summary. The proposed technique shows good performance but suffers from the issues of anaphora and co-resolution.

In 2018, Rathod illustrated a graph-based approach for ranking sentences on a dataset of Marathi news documents to generate extractive summaries [26]. Two approaches, similarity-based and position-based, were implemented. The results of the similarity based approach were more accurate compared to the position based approach. Sarwadnya and Sonawane presented a multi-document extractive graph-based summarization model on Marathi dataset [27]. Kanitha et al. illustrated an extractive summary generation methodology for summarizing Malayalam language news articles using graph-based technique. The nodes represented sentences and the relations between sentences were indicated by edges [28]. The results obtained for extractive summarization were good. Ghosh et al. illustrated automatic summarization of Bangla news articles using graph-based technique with sentence scoring [29]. Their system outperformed the existing systems available for summarizing Bangla news articles.

Current ML approaches use a popular concept of word embeddings, wherein words are represented by vectors. The vectors capture latent knowledge about a language, like semantics or word analogies [30, 31]. Word2Vec, Bidirectional Encoder Representations from Transformers (BERT), Global Vectors (GloVe) and fastText are examples of popular models for word vectors; however, fastText is the only model that provides pre-trained word vectors in Konkani. These are trained on documents available on Common Crawl and Wikipedia [30],[31]. Since the model is pre-trained on Konkani documents, the word embeddings based approach using graph-based method is a language dependent method. Word embeddings are used for summarization along with graph-based method; however, using such word embeddings is restricted to the availability of the pre-trained embeddings in a language under consideration. If a

Just Accepted

sufficiently large dataset is available, then fastText can be trained on the dataset. Konkani, being a low-resource language, a large dataset was unavailable at the time this research was carried out.

Phung and De Vine proposed a model that combined the use of PageRank algorithm and word embeddings for performing extractive summarization of Vietnamese texts [32]. The results of their experiment showed an improved performance of the system when word embeddings were used for sentence representation. Using sentence embeddings has shown to improve the performance of systems in keyword or topic extraction task. In the work presented by Zuo et al., word embedding technique using Word2Vec embeddings was used along with TextRank algorithm for extracting keywords or topics from text documents [33]. The experiments showed that using the combination of Word2Vec and TextRank improved the performance of the system.

A good representation of an input document is a key to developing a good summarization system. Word embeddings work well in learning sentence representations. Al-Sabahi et al. proposed an approach using modified BM25 with word embeddings for constructing sentence vectors from word vectors [34]. First, they represented the entire document as a set of sentence vectors and then calculated the similarity between each pair of sentence vectors. They used TextRank to rank the sentences and selected the top ranked sentences to generate the summary. This model performed better when compared to other state-of-the-art approaches.

A good quality sentence similarity graph serves as a foundation for a quality graph based extractive summarization system. Jiang et al. proposed an extractive summarization approach using deep embeddings features, Word2Vec and BERT. The performance of the proposed approach surpassed that of 'tf-idf' approach and achieved state-of-the-art results [35]. Another approach, presented by Moradi et al., used word embeddings for summarizing biomedical text documents [36]. They pre-trained the neural language models on a large dataset of biomedical documents. The system first constructed a graph of input text and thereafter used the domain specific vector representations to determine the strength of the relationship between various sentences. They also evaluated various techniques used for graph ranking. The experimental results pointed that the system can outperform other techniques when appropriate combinations of embeddings are used with an effective ranking approach.

Using word embeddings for graph-based extractive summarization is seen in the work presented in some Indian languages. Manju et al. used fastText word embeddings with graph based approach, TextRank, for performing multi-document summarization of Malayalam text documents [37]. Chandrika and Kallimani used word embeddings with PageRank algorithm for performing extractive summarization of Kannada text documents [38].

In summary, we observe that language independent graph-based approaches have been extensively used for summarization of text documents and also been used for performing automatic text summarization in Indian languages like Bangla, Marathi, Hindi, Malayalam, etc. This approach is preferred for its ability to produce promising summaries without needing training corpus. However, the language dependent word embeddings based approach using graph-based method shown promising results. Using word vectors boosts the performance, as the word vectors can pick up latent information from a language. When focusing on the languages on which the said approached has been applied on, we see that the languages of preference are English, and some Indian languages like Malayalam and Kannada. Although, there is a scope for further exploring the application of language dependent approach using word embeddings with graph-based method in Konkani. A comparison of the two approaches will definitely help

Just Accepted

comprehend the pros and cons of each of the approaches and gain insights on further improvement of the applied methods. The primary aim of this work is to understand the impact similarity measures have on the graph construction and on the text summarization task on our dataset.

3 GRAPH-BASED AUTOMATIC TEXT SUMMARIZATION IN KONKANI

The algorithm used for Graph-based ATS in Konkani is explained in this section along with the details of the dataset used for the experiments. Thereafter, the variations have been proposed in the similarity measures to help ascertain the depth of relationship between sentences.

3.1 The Algorithm

The Konkani text documents are taken as input and a graph is created for each document with sentences of the document forming its vertices. A fully connected graph is created, and each edge is given a weight that is calculated using similarity measures. A method, similar to the one proposed by Barrios et al. [39], has been employed to evaluate various sentence similarity measures to weigh the edges of the graph. Given below are the steps applied in the summary generation process. Figure 1 illustrates the summary generation steps.

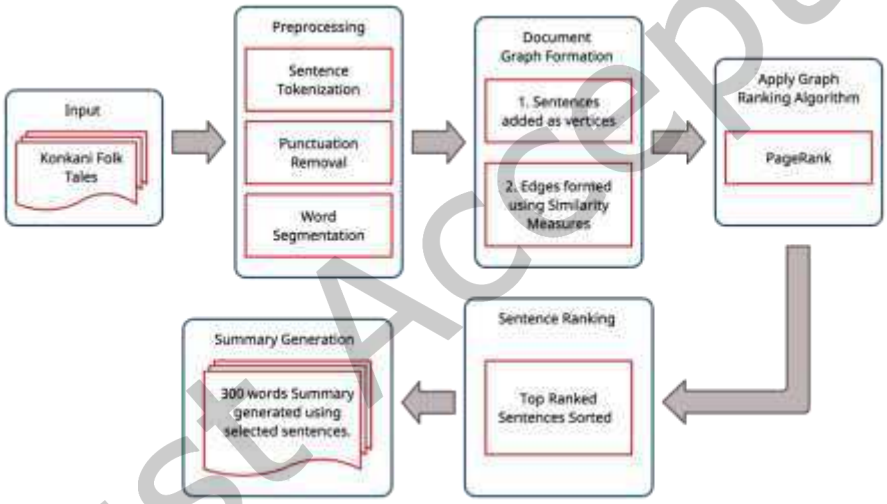


Figure 1: Schematic illustration of the summary generation process.

1) Pre-processing

Input text is tokenized into smaller units of text. In this case, the units are sentences that compose a folktale. Thereafter, the sentences are "cleaned", eliminating any punctuation. However, the summary that is eventually generated keeps the punctuations. Thereafter, the words are segmented from the sentences.

2) Graph Formation

A fully connected undirected graph of the text units is constructed, wherein the text units form the vertices of the graph. The vertices are then connected to other vertices of the document based on their relationship. Hence, the

Just Accepted

relationships between different sentences form the edges of this graph. Weights can then be assigned to the edges, depending on the strength of the relationship between the sentences (or vertices). The relationships are defined using similarity measures. An example of a fully connected undirected graph generated using Common tokens similarity measure with five sentences from the Konkani dataset is given in Figure 2. The five sentences are numbered from 0 to 4.

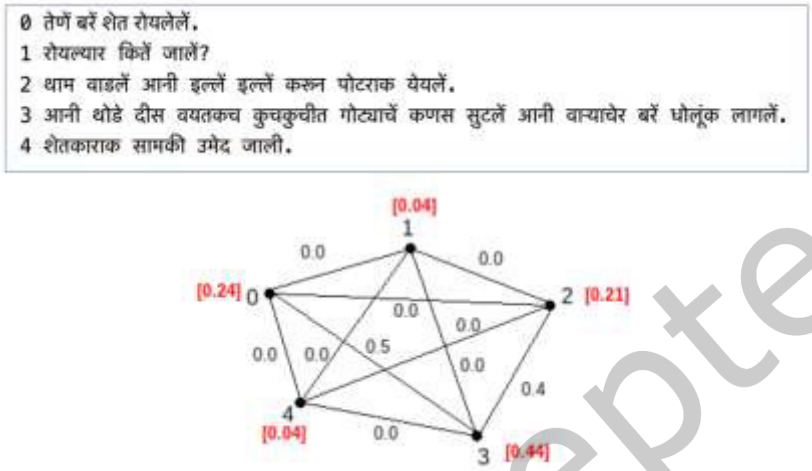


Figure 2: An example of graph formation using five Konkani sentences

3) Sentence Ranking

"PageRank" algorithm is used to rank the scores of vertices in the graph; however, any other graph ranking algorithm could also be used, provided the graph ranking algorithm under consideration can consider the weighted edges [18], [40]. Next, every vertex is assigned a ranking score with the help of "PageRank" algorithm. The vertices are then arranged in the decreasing order of their scores.

PageRank algorithm, proposed by Brin et al. [12], was originally designed for the analyses of web links. The importance of a page is determined by the number of links leading to it. Equation (1) is used to calculate the "PageRank" score of a vertex 'V' [18],

$$PR\,(V_i) = (1-d) + d * \sum_{V_j \epsilon\, \ln(V_j)} \frac{PR\,(V_j)}{|Out(V_j)|} \qquad (1)$$

In equation (1), 'd' is the damping factor, the value of which can be between 0 and 1 (value of 'd' is typically set at 0.85) [12]. PageRank algorithm computes the score by assessing the impact of both incoming and outgoing links. Traditionally, PageRank was developed to work on directed graphs; although an undirected graph can be converted to a directed graph by adding two edges, each in opposite directions for every undirected edge in the graph. Given the graph formation, the time complexity for PageRank to execute on the graph would be $O((n\,(S+R))$, where 'n' is the total count of iterations, 'S' are the sentences or graph nodes and 'R' is the relationship between sentences. The space complexity using an adjacency matrix would be $O(S^2)$.

Just Accepted

4) Sentence Ranking

The next step is determining which sentences can be a part of the summary. The sentences are ranked in descending order of their scores obtained after applying the PageRank algorithm.

5) Summary Generation

The top ranking sentences are then picked to compose the document's summary. The word count of the summary is restricted to a user-defined threshold; in this case, it is 300 words for the intent of evaluation.

3 KONKANI DATASET

Konkani language is spoken along the 'Konkan coast' that is the west coast of India. There are many dialects of Konkani spoken along various regions of the coast. The dialect considered for this study is used in the state of Goa.

The genre of the dataset is Konkani literature with a focus on folktales. Folktales are short stories that have a cultural significance and they were traditionally passed down from a generation to the next verbally. These folktales have a moral or a good teaching that is taught to the younger generations. We have compiled a dataset on Konkani language literature to facilitate research in the said language for automatic summarization or any allied areas of NLP [3].

The source documents of the dataset were collected from 5 books featuring folktales written in Konkani. These folktales are based on various locations from India, Russia and Goa. Some stories were originally in Russian and were later translated to Konkani. These books, written in Devanagari script, by various authors are rare and were difficult to procure. The dataset captures the essence of Konkani literature, specifically from the state of Goa, with its 71 unique folk tales. This dataset comprises 9849 sentences, with each story having an average of 138 sentences and 1155 words. The number of sentences in the shortest story of the dataset is 520 words and the longest story has 2852 words.

For every folktale, a 300-word abstract was created by two independent subject experts for the evaluation of the system generated summaries. The length of the system generated summaries was capped at 300 words. This 300 words limit was set after consulting the subject experts, since 300 words make up a page of a document. Also, the length of the shortest story was taken into consideration, which is 520 words, while setting the word limit. These 'gold-standard' human generated summaries were used for the evaluation of the quality of the system generated summaries. The detailed process of the compilation of the dataset can be found in [3].

4 TAXONOMY OF SIMILARITY MEASURES

Similarity measures are used to ascertain the depth of relationship between sentences. In this paper, several similarity measures were examined to compute the similarities between sentences [18, 39].

Similarity measures play an important role and are actually orthogonal to the graph-based ranking model. Since we make use of weighted fully connected graphs, the output of the ranking algorithm directly depends on how those weights are assigned to the edges. This influences which sentences get ranked higher or lower, and finally, which of them gets to constitute the final summary.

The similarity measures used in this experiment are grouped into three categories, viz. language independent, i.e. Word-level similarity measures and Character-level similarity measures, and, language dependent Word embedding based similarity measures. We have compared three word-level similarity measures that work with words in the

Just Accepted

sentences, namely Common Tokens, Cosine Measure, Jaccard Measure and three character-level similarity measures, Jaccard Measure, Levenshtein Sentence Weight and Jaro–Winkler Distance, that work by considering characters present in a sentence. Also, we have four word embedding based similarity measures, Cosine Distance, Euclidean Distance, City Block Distance and Word Mover's Distance. The descriptions of the measures is provided below.

4.1 Word-level Similarity Measures

- Common Tokens

It calculates the number of common tokens or overlaps between two sentences Si and Sj, given by equation (2) below. This measure was the original similarity measures used in "TextRank" [14],

$$Similarity(S_i, S_j) = \frac{|\{w_k | w_k \epsilon\, S_i \& w_k \epsilon S_j\}|}{log\,(|S_i|) + log\,|(S_j|)} \qquad (2)$$

- Cosine Metric

If we have two sentences, viz., A and B, then the 'Ochiai coefficient' (K) of A and B is outlined in equation (3) [41],

$$K = \frac{|A \cap B|}{\sqrt{|A|.|B|}} \qquad (3)$$

In case where one of A and B is an empty set, then K = 0, and K=1 if both are empty sets.

- Jaccard Metric

This measure is computed assuming sentences A and B are independent word groups [41]. Equation (4) defines Jaccard measure as,

$$J = \frac{|A \cap B|}{|A \cup B|} \qquad (4)$$

If A and B are empty sets, then J=1.

4.2 Character-level Similarity Measures

- Levenshtein Sentence Weight

'Levenshtein Distance' is a prerequisite to calculating 'Levenshtein Measure'. Levenshtein Distance is the count of the minimum number of edits on a string when transforming it into another. The Levenshtein Distance of two strings 'c' and 'd' of length |c| and |d| respectively is defined as levc,d(|c|,|d|) given by equation (5) as follows [42],

$$lev_{c,d}(g,h) = \begin{cases} \max(g,h) \\ \min\begin{cases} lev_{c,d}(g-1,h) + 1 \\ lev_{c,d}(g,h-1) + 1 \\ lev_{c,d}(g-1,h-1) + 1_{c_g \neq d_h} \end{cases} \end{cases} \qquad (5)$$

Once Levenshtein Distance is obtained, the Levenshtein Sentence Weight (LSW) [32] is calculated using the following equation (6),

$$LSW\,(S_i, S_j) = \frac{maxlength\,(S_i,S_j) - lev(S_i,S_j)}{maxlength\,(S_i,S_j)} \qquad (6)$$

Just Accepted

- Jaro–Winkler Distance

This method was proposed by Winkler [44] over the original, proposed by Jaro [45], [46]. Given two strings S1 and S2, the Jaro–Winkler distance '*dw*' is given by equation (7) as shown below, where '*dj*' is 'Jaro distance' of S1 and S2, '*l*' is the common prefix length at the beginning of the string to a maximum of four characters and '*p*' is the 'scaling factor', which is usually set to 0.1.

$$d_w = d_j + (lp(1 - d_j)) \quad (7)$$

4.3 fastText Word Embedding based Similarity Measures

'fastText' is open source and provides text representations of words [30], [47]. The word vectors have a fixed size dimension of 300. This model can be trained on any dataset if it is large enough. However, this is constrained by the availability of a large computational power and a sufficiently large dataset. For some of the languages that lack the availability of large datasets, 'fastText' provides pre-trained word embeddings, and they are made available in 157 languages, including Konkani. These pre-trained word embeddings were trained on Common Crawl and Wikipedia articles [31]. The models were trained with the Continuous-Bag-Of-Words (CBOW) model with 300 dimension, length of character n-grams as 5, window size of 5 and 10 negatives.

'fastText' pre-trained word embeddings can predict Out-Of-Vocabulary (OOV) words, i.e. it can generate embeddings for words that were not "seen" at the time of training. This is a very critical point for this work since the dataset used for training 'fastText' model was completely different from the one used for summary generation. Therefore, the likelihood of the model encountering OOV words was significantly higher. It generates word embeddings of the previously unseen words by constructing a vector obtained by the summation of the sub-vectors that constitute the OOV word. Also, it can generate word embeddings for rare words, as it takes into consideration the internal word structure.

In this study, we use Cosine Distance, Euclidean Distance, City Block Distance and Word Mover's Distance to compute similarity measures using word embeddings. The word embedding models provide a vector for each of the words and not each sentence. Word Mover's Distance uses word embeddings [48, 49, 50]. Cosine Distance, Euclidean Distance and City Block Distance are computed between two vectors. Therefore, these multiple word vectors of words in a sentence need to be reduced to a single representational sentence vector. The commonly used approach for reducing multiple vectors to single vector is to compute an average of all the word embeddings to get the vector representation of the sentence. The four similarity measures for word embeddings are described below.

- Cosine Distance: Cosine distance between input vector u and input vector v is defined in equation (8).

$$\text{Cosine Distance} = 1 - \frac{u \cdot v}{\|u\|_2 \|v\|_2} \quad (8)$$

- Euclidean Distance: Euclidean distance is a straight line distance between any two data points in a given plane. In our case, it computes distance between two vectors. If u and v are two vectors, then the Euclidean distance between the two is given by equation (9).

$$\text{Euclidean Distance} = \|u - v\|_2 \quad (9)$$

Just Accepted

- City Block Distance: City Block Distance, also known as Manhattan Distance, is calculated between two vectors u and v, given by equation (10).

$$\text{City Block Distance} = \sum_i |u_i - v_i| \quad (10)$$

- Word Mover's Distance: The Word Mover's Distance (WMD) takes into consideration the similarities in words in the word embedding space. WMD uses the word embeddings of the words in two sentences to calculate the minimum distance the words in one sentence need to "travel" in a semantic space to get to the words in the other sentence. The method of 'Word Mover's Distance' was introduced by Matt Kusner et al., and was inspired by the Earth Mover's Distance [48, 49, 50]. WMD computes the distance between two sentences even if there are no common words between them. It formulates the distance as an optimization of a transportation problem between embedded words of the sentences. It is, therefore, the minimum distance required to transport words present in one sentence to another.

5 EVALUATIONS AND RESULTS

We examined language independent similarity measures and language dependent similarity measures using graph-based approach for performing automatic text summarization of Konkani texts. A fully connected directed graph was constructed and PageRank algorithm was used for ranking the sentences according to their scores and the top ranking sentences were selected to form the summary. The language dependent similarity measures used fastText pre-trained word embeddings that provided vector representation of words.

We use 'ROUGE' toolkit to perform the assessments on the system results. 'ROUGE' has proven to work well with human evaluations and uses N-gram statistics [51]. The generated summary of 300 words is compared with two human-generated summaries [3]. ROUGE metrics are used to measure the overlap of uni-grams, bi-grams and 'Longest Common Subsequence' (LCS) between the human-generated summaries and the system generated summaries. ROUGE-1 represents uni-gram scores, ROUGE-2 shows bi-gram scores, and ROUGE-L depicts LCS scores. These measures are used to check the fluency of the generated summaries, with the increasing levels of granularity moving from uni-grams to bi-grams to LCS. If the word-orderings in the summary generated summary are similar to the human-generated summary, it indicates the fluency of the summary generated by the system. The details of the evaluation metrics are highlighted in [51]. The tables below show the scores of various similarity measures obtained by evaluating the system-generated summaries with both the corresponding human summaries. We also introduce a new benchmark which comprises a 300-word summary created using sentences marked by subject experts as sentences which are essential and to be included in a summary. This benchmark and other Baselines, like Modified Luhn's Heuristic Baseline and Lead Baseline, were used to compare and analyse the performance of the implemented systems [52]. Luhn's Heuristic technique is a statistical method used for extractive summarization [53]. The modification of the Luhn's Heuristic technique, used in this paper, uses 30 relevant keywords chosen using YAKE [54]. The scores of the sentences are calculated by squaring the total count of keywords and dividing this value by the window size [53]. The window size is the maximum distance between two keywords. The sentences with the highest scores then form the summary. A Lead Baseline summary is generated using the first 300 words appearing in a text document. It was used by Document Understanding Conferences (DUC) and has been a relatively tough to outperform regardless of its simplicity.

Table 1 displays the compilation of ROUGE-1 uni-gram scores. Table 2 presents the ROUGE-2 bi-gram scores. Table 3 illustrates ROUGE-L (LCS) scores. ROUGE-1 scores are higher than ROUGE-2 and ROUGE-L, because a matching

Just Accepted

term in ROUGE-2 will also be in ROUGE-1, but it's not true the other way round. And, ROUGE-L scores are higher than ROUGE-2, as ROUGE-L does not check for consecutive word matches but it checks for in-sequence word matches and it naturally incorporates longest common n-grams in a sequence. Each of the similarity measures was measured along with PageRank algorithm and represented in the 'System' column. In each table, we have computed the corresponding ROUGE metric, where 'Precision' and 'Recall' provide a quantitative analysis of the overlap between the system-generated summaries and the human-generated summaries [55].

Precision attempts to identify how much of the system summary is actually important. This is necessary because, despite producing a summary, the contents could be unimportant. Recall tries to identify how much of the contents of the gold-standard human-generated summary have been sufficiently captured by the system-generated summary. The F-Score considers both Precision and Recall and then reports them together as one score. The scores of Precision, Recall and F-score are in the range between 0 and 1; a score of '0' indicates that there is no overlap between human generated summary and system generated summary and a score of '1' means that there is a strong overlap between the two summaries.

Table 1: ROUGE-1 uni-gram scores

System	ROUGE-1 (uni-gram)		
	Precision	Recall	F-Score
PageRank-CommonTokens	0.32849	0.32823	0.32834
PageRank-CosineMetric	0.32619	0.32543	0.32579
PageRank-JackardsMetric	0.32317	0.32218	0.32266
PageRank-JaroWinklerMetric	0.31666	0.31632	0.31647
PageRank-LevenshteinDistance	0.31173	0.31254	0.31212
PageRank-LevenshteinSentenceWeight	0.31734	0.31590	0.31660
PageRank-FastText-Cosine-Distance	0.29315	0.29335	0.29323
PageRank-FastText-Euclidean-Distance	0.29957	0.29848	0.29900
PageRank-FastText-City-Block-Distance	0.29973	0.29865	0.29917
PageRank-FastText-Word-Movers-Distance	0.29730	0.29664	0.29695
Lead-Baseline-300-words	0.30165	0.30147	0.30154
Modified-Luhns-Heuristic-Baseline	0.31349	0.31421	0.31383
Human-Annotated-Benchmark	0.35460	0.35844	0.35608

Just Accepted

Table 2: ROUGE-2 bi-gram scores

System	ROUGE-2 (bi-gram)		
	Precision	Recall	F-Score
PageRank-CommonTokens	0.08688	0.08680	0.08683
PageRank-CosineMetric	0.08453	0.08438	0.08445
PageRank-JackardsMetric	0.08145	0.08124	0.08134
PageRank-JaroWinklerMetric	0.07242	0.07235	0.07238
PageRank-LevenshteinDistance	0.08085	0.08104	0.08094
PageRank-LevenshteinSentenceWeight	0.07591	0.07561	0.07575
PageRank-FastText-Cosine-Distance	0.06968	0.06977	0.06972
PageRank-FastText-Euclidean-Distance	0.06849	0.06827	0.06838
PageRank-FastText-City-Block-Distance	0.06919	0.06897	0.06908
PageRank-FastText-Word-Movers-Distance	0.07138	0.07125	0.07131
Lead-Baseline-300-words	0.08103	0.08097	0.08099
Modified-Luhns-Heuristic-Baseline	0.07876	0.07889	0.07882
Human-Annotated-Benchmark	0.10908	0.11088	0.10977

Table 3: ROUGE-L (LCS) scores

System	ROUGE-L (LCS)		
	Precision	Recall	F-Score
PageRank-CommonTokens	0.31984	0.31960	0.31970
PageRank-CosineMetric	0.31828	0.31755	0.31790
PageRank-JaccardsMetric	0.31587	0.31491	0.31537
PageRank-JaroWinklerMetric	0.31086	0.31052	0.31067
PageRank-LevenshteinDistance	0.30389	0.30468	0.30427
PageRank-LevenshteinSentenceWeight	0.31264	0.31123	0.31192
PageRank-FastText-Cosine-Distance	0.28903	0.28924	0.28912
PageRank-FastText-Euclidean-Distance	0.29600	0.29493	0.29544
PageRank-FastText-City-Block-Distance	0.29625	0.29518	0.29570
PageRank-FastText-Word-Movers-Distance	0.29346	0.29281	0.29312

Just Accepted

Lead-Baseline-300-words	0.29659	0.29642	0.27648
Modified-Luhns-Heuristic-Baseline	0.30753	0.30822	0.30786
Human-Annotated-Benchmark	0.34847	0.35228	0.34994

It can be observed that the language independent word-level similarity measure, 'PageRank-CommonTokens', has scored the highest in all the three metrics, ROUGE-1, ROUGE-2 and ROUGE -L; this indicates that the measure proposed by R. Mihalcea performs very well [18]. The results of other measures are also satisfactory, but we noticed that the language independent word-level measures perform better compared to language independent character-level measures. The language independent word-level measures take into consideration the number of common words between two sentences, which results in a better representation of similarity between the sentences as compared to going down to the lowest level of characters.

Amongst the language dependant similarity measures, City-Block-Distance performed the best. 'fastText' based approaches that use average of word embeddings, i.e. Cosine distance, Euclidean distance and City Block distance, could perform better if stop-word removal and stemming were performed. This is also the case with language dependent similarity measure, Word Movers distance, where stop word removal can greatly impact its performance. Unfortunately, Konkani, being a low-resource language, these steps could not be performed. 'fastText' embeddings are trained on Wikipedia articles, which are not in large numbers for Konkani, as compared to other languages. Also, Wikipedia articles are not from the same genre as the dataset documents used in this study. 'fastText' pre-trained embeddings have representations for 292,012 unique words, and our dataset has 14,068 unique words. Out of these, only 6938 words are directly represented in the fastText pre-trained embeddings, which is very low. Although fastText can handle OOV words using n-grams, the number of directly mapped words is too low, and this, contributes to the lower performance of the word embeddings' approach. Therefore, it is desirable to have an embedding trained on data from a similar genre with a larger representation of words present in the dataset.

When comparing the systems with the baselines, all the language independent word-level systems could beat the baselines and two language independent character-level systems, Jaro-Winkler and Levenshtein Sentence Weight, could also beat the baselines. We observed that graph-based techniques can produce relatively fluent system summaries when compared with human-generated summaries produced for this dataset. This shows that the unsupervised graph-based techniques can produce acceptable system summaries in the absence of training data, using language independent word-level similarity measures.

6 CONCLUSION

We performed extractive text summarization of Konkani language texts using graph-based methodology for examining language independent approach and language dependent approach, using various similarity measures in each of the approaches. The text summarization method uses an undirected graph, in which the sentences form the nodes, and the edges that connect these nodes define the relationship between the sentences. These relationships are dependent on the use of similarity measures. The language independent word and character-level similarity measures do not use any language specific resources for Konkani, such as stop-word list. The language dependent similarity measures used fastText word embeddings, pre-trained on Konkani texts. The summaries produced by the two approaches were then evaluated by ROUGE toolkit.

Just Accepted

Amongst the language independent similarity measures, we can see that the word-level measures, such as Common Token, Cosine metric and Jaccard Metric have performed better than the other character-level measures. Amongst language dependant similarity measures, City Block Distance performed the best. However, none of the systems could beat the Human Annotated Benchmark, and, only five language independent similarity measures, viz. Common tokens, Cosine metric, Jaccard metric, Jaro-Winkler Distance and Levenshtein Sentence Weight, could beat the Baselines.

An interesting aspect is that the Graph-based similarity measures performed very well despite not using any form of language dependent tools for stop-words removal, lemmatization or any training corpora. This has shown promising results with our Konkani dataset. Language independent similarity measures are important when considering low-resource languages, such as Konkani. There are many such low-resource languages and promising results can be obtained for extractive summarization of documents in other similar languages by using language independent similarity measures presented in this study.

As future work, expanding the study to other low-resource languages can be undertaken using the graph-based approach with language independent and language dependent similarity measures. The language dependent approach implemented in this work can be improved by training the fastText word embeddings on the same genre of the dataset if a sufficiently large dataset is available. Also, the impact of stop-word removal and stemming on the summarization process can be examined if these tools are available in the concerned language. It can be noted that graph-based systems perform well because graphs consider the global context of the documents in question, with sentences recommending other sentences of importance, making this model not only flexible but also a powerful tool for automatic text summarization.

Just Accepted

REFERENCES

[1] Narendra Andhale and Laxmi A. Bewoor. 2016. An overview of Text Summarization techniques. International Conference on Computing Communication Control and Automation (ICCUBEA). 10.1109/iccubea.2016.7860024.

[2] N. Moratanch and S. Chitrakala. 2017. A survey on extractive text summarization. International Conference on Computer, Communication and Signal Processing (ICCCSP).10.1109/icccsp.2017.7944061.

[3] Jovi D'Silva and Uzzal Sharma. 2019. Development of a Konkani Language Dataset for Automatic Text Summarization and its Challenges. International Journal of Engineering Research and Technology. International Research Publication House, Vol 12, No 10. ISSN 0974-3154, 1813-1817.

[4] Vishal Gupta. 2013. A Survey of Text Summarizers for Indian Languages and Comparison of their Performance. Journal of Emerging Technologies in Web Intelligence, Vol 5, No 4, 361-366.

[5] Ketakee Nimavat and Hetal A. Joshiara.2017. Query-Based Summarization Methods for Conversational Agents: An Overview. International Journal of Advanced Research in Computer Science, Vol 8 No 8, ISSN No. 0976-5697.

[6] Jovi D'Silva and Uzzal Sharma.2019. Automatic Text Summarization of Indian Languages: A Multilingual Problem. Journal of Theoretical and Applied Information Technology, Vol. 97, No. 1, ISSN: 1992-8645.

[7] Hmida Firas. 2014. Language Independent Summarization Approaches. Innovative Document Summarization Techniques: Revolutionizing Knowledge Understanding 2014, LINA Nantes-University, France. 10.4018/978-1-4666-5019-0.ch013.

[8] Ahmed A. Saleh and Li Weigang. 2017. Language independent text summarization of western European languages using shape coding of text elements. 13th International Conference on Natural Computation. Fuzzy Systems and Knowledge Discovery (ICNC-FSKD) 2017. 10.1109/fskd.2017.8393116.

[9] Ahmad T. Al-Taani. 2017. Automatic text summarization approaches. International Conference on Infocom Technologies and Unmanned Systems, Trends and Future Directions (ICTUS). 10.1109/ictus.2017.8285983

[10] S. Rahimi, Ali T. Mozhdehi and Mohamad Abdolahi. 2017. An overview on extractive text summarization. IEEE 4th International Conference on Knowledge-Based Engineering and Innovation (KBEI). 10.1109/kbei.2017.8324874.

[11] Document Understanding Conferences (DUC), National Institute of Standards and Technology (NIST), Available: https://www-nlpir.nist.gov/projects/duc/guidelines/2002.html.

[12] Sergey Brin and Lawrence Page. (1998). The anatomy of a large-scale hypertextual Web search engine. In Computer Networks and ISDN Systems, Vol. 30, 1-7. https://doi.org/10.1016/S0169-7552(98)00110-X.

[13] Statement –4: Distribution of Population by Schedule and Other Languages India, States and Union Territories – 2011, Office of the Registrar General & Census Commissioner, India, Ministry of Home Affairs, Government of India, 2011, https://censusindia.gov.in/2011Census/Language-2011/Statement-4.pdf. Accessed November 15, 2020.

[14] Rada Mihalcea and Paul Tarau. 2004. TextRank: Bringing Order into Texts. Conference on Empirical Methods in Natural Language Processing, Barcelona, Spain. http://acl.ldc.upenn.edu/acl2004/emnlp/pdf/Mihalcea.pdf.

[15] Gunes Erkan and Dragomir R. Radev. 2004. LexRank: Graph-Based Lexical Centrality as Salience in Text Summarization. Journal of Artificial Intelligence Research 22, 457–479. https://doi.org/10.48550/arXiv.1109.2128.

[16] Rafael Ferreira, Frederico Freitas, Luciano De Souza Cabral, Rafael Dueire Lins, Rinaldo Lima, Gabriel França, Steven J. Simskez and Luciano Favaro. 2013. A Four Dimension Graph Model for Automatic Text Summarization. IEEE/WIC/ACM International Conferences on Web Intelligence (WI) and Intelligent Agent Technology (IAT), 389-396.

[17] Ohm Sornil and Kornnika Gree-ut. 2006. An Automatic Text Summarization Approach using Content-Based and Graph- Based Characteristics. Cybernetics and Intelligent Systems, IEEE Conference 2006, 1–6. http://dx.doi.org/10.1109/ICCIS.2006.252361.

[18] Rada Mihalcea. 2004. Graph-based ranking algorithms for sentence extraction, Applied to text summarization. Proceedings of the ACL 2004 on Interactive poster and demonstration sessions, 20. https://doi.org/10.3115/1219044.1219064.

[19] Jovi D'Silva and Uzzal Sharma.2022. Explorations in Graph-based Ranking Algorithms for Automatic Text Summarization on Konkani Texts. International Conference on Sustainable Advanced Computing- ICSAC 2021. Sustainable Advanced Computing, Chapter 4. 10.1007/978-981-16-9012-9_4.

[20] Hui Lin, Jeff Bilmes and Sasha Xie. 2009. Graph-based submodular selection for extractive summarization. IEEE Workshop on Automatic Speech Recognition & Understanding. 10.1109/asru.2009.5373486.

[21] Chirantana Mallick, Ajit K. Das, Madhurima Dutta, Asit K. Das and Apurba Sarkar. 2018. Graph-based Text Summarization Using Modified TextRank, Soft Computing in Data Analytics, 137-146. 10.1007/978-981-13-0514-6_14.

[22] Munisamy Banu, C. Karthika, P. Sudarmani and T. V. Geetha. 2007. Tamil Document Summarization Using Semantic Graph Method. International Conference on Computational Intelligence and Multimedia Applications (ICCIMA 2007). 2007 IEEE. DOI: 10.1109/ICCIMA.2007.247.

[23] Sankar K., Vijay Sundar Ram R. and Sobha Lalitha Devi. 2011. Text Extraction for an Agglutinative Language. Kommaluri Vijayanand, and L. Ramamoorthy, (ed), Language In India, Problems of Parsing in Indian Languages, Vol. 11, ISSN 1930-2940, 56-59.

[24] K. Vimal Kumar, Divakar Yadav and Arun Sharma. 2015. Graph Based Technique for Hindi Text Summarization. Information Systems Design and Intelligent Applications, 301–310. 10.1007/978-81-322-2250-7_29.

[25] Vipul Dalal and Latesh Malik. 2018. Semantic Graph Based Automatic Text Summarization for Hindi Documents Using Particle Swarm Optimization. ICTIS 2017: Information and Communication Technology for Intelligent Systems (ICTIS 2017), Vol. 2, 284-289.

[26] Yogeshwari V. Rathod. 2018. Extractive Text Summarization of Marathi News Articles. International Research Journal of Engineering and Technology (IRJET), e-ISSN: 2395-0056, p-ISSN: 2395-0072, Vol 05, Issue: 07.

[27] Vaishali Sarwadnya and Sheetal Sonawane. 2018. Marathi Extractive Text Summarizer Using Graph Based Model. Fourth International Conference on Computing Communication Control and Automation (ICCUBEA), IEEE. 10.1109/ICCUBEA.2018.8697741.

[28] Kanitha D. K., M. D. Muhammad Noorul and S. A. Shanavas. 2018. Malayalam Text Summarization Using Graph Based Method. International Journal of Computer Science and Information Technologies (IJCSIT), Vol. 9, No. 2, 40-44. ISSN: 0975-9646.

[29] Partha Ghosh, Rezvi Shahariar and Muhammad A. Khan. 2018. A Rule Based Extractive Text Summarization Technique for Bangla News Documents. International Journal of Modern Education and Computer Science, Vol 10, 44-53. 10.5815/ijmecs.2018.12.06.

[30] Piotr Bojanowski, Edouard Grave, Armand Joulin and Tomas Mikolov.2017. Enriching Word Vectors with Subword Information. Transactions of the Association for Computational Linguistics, Vol. 5, Cambridge, MA, 135-146. 10.1162/tacl_a_00051.

[31] Edouard Grave, Piotr Bojanowski, Prakhar Gupta, Armand Joulin and Tomas Mikolov. 2018. Learning Word Vectors for 157 Languages. Proceedings of the International Conference on Language Resources and Evaluation (LREC 2018). https://doi.org/10.48550/arXiv.1802.06893.

[32] Viet Phung and Lance De Vine. 2015. A Study on the Use of Word Embeddings and PageRank for Vietnamese Text Summarization. Proceedings of the 20th Australasian Document Computing Symposium. 10.1145/2838931.2838935.

[33] Xiaolei Zuo, Silan Zhang and Jingbo Xia. 2017. The enhancement of TextRank algorithm by using word2vec and its application on topic extraction. Journal of Physics: Conference Series, Vol. 887, The 2nd Annual International Conference on Information System and Artificial Intelligence (ISAI2017), Vol. 887, China.

[34] Kamal Al-Sabahi and Zhang Zuping. 2019. Document Summarization Using Sentence-Level Semantic Based on Word Embeddings. International Journal of Software Engineering and Knowledge Engineering, Vol. 29, No. 2, 177–196. https://doi.org/10.1142/S0218194019500086.

[35] Zhuolin Jiang, Manaj Srivastava, Sanjay Krishna, David Akodes and Richard Schwartz. 2020. Combining Word Embeddings and N-grams for Unsupervised Document Summarization.

[36] Milad Moradi, Maedeh Dashti and Matthias Samwald. 2020. Summarization of biomedical articles using domain-specific word embeddings and graph ranking. Journal of Biomedical Informatics, Vol. 107, ISSN 1532-0464. https://doi.org/10.1016/j.jbi.2020.103452.

[37] K. Manju, David Peter and Sumam Mary Idicula. 2021. A Framework for Generating Extractive Summary from Multiple Malayalam Documents. Information 2021, Vol. 12, 41. https://doi.org/10.3390/info12010041.

Just Accepted

[38] Chandrika C. Prasad., Jagdish S. Kallimani. 2022. Extractive Text Summarization of Kannada Text Documents Using Page Ranking Technique. In: Hemanth D.J., Pelusi D., Vuppalapati C. (eds) Intelligent Data Communication Technologies and Internet of Things. Lecture Notes on Data Engineering and Communications Technologies, Vol. 101, Springer, Singapore. https://doi.org/10.1007/978-981-16-7610-9_51.

[39] Federico Barrios, Federico López, Luis Argerich and Rosa Wachenchauzer. 2015. Variations of the Similarity Function of TextRank for Automated Summarization. Presented at the Argentine Symposium on Artificial Intelligence (ASAI) 2015 - 44 JAIIO, 44 JAIIO - ASAI 2015 - ISSN: 2451-7585, 65-72. https://doi.org/10.48550/arXiv.1602.03606.

[40] Rada Mihalcea and Dragomir R. Radev. 2011. Graph-based natural language processing and information retrieval. Cambridge: Cambridge University Press.

[41] Donald A. Jackson, Keith M. Somers and Harold H. Harvey. 1989. Similarity coefficients: measures of co-occurrence and association or simply measures of occurrence? The American Naturalist, Vol. 133, 436-453. https://doi.org/10.1086/284927.

[42] V. I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. Soviet physics doklady, Vol. 10, No. 8.

[43] Sankar K. and Sobha Lalithadevi.2009. An approach to text summarization. In Proceedings of the Third International Workshop on Cross Lingual Information Access: Addressing the Information Need of Multilingual Societies, Association for Computational Linguistics, 53-60.

[44] William E. Winkler. The state of record linkage and cur- rent research problems. Statistics of Income Division, Internal Revenue Service Publication.

[45] Matthew A. Jaro. 1989. Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida. Journal of the American Statistical Association, Vol. 84, No. 406, 414–420. https://doi.org/10.2307/2289924.

[46] Matthew A. Jaro. 1995. Probabilistic linkage of large public health data files (disc: P687-689). Statistics in Medicine, Vol. 14, 491–498. https://doi.org/10.1002/sim.4780140510.

[47] Jovi D'Silva and Uzzal Sharma. 2022. Automatic text summarization of Konkani texts using pre-trained word embeddings and deep learning. International Journal of Electrical & Computer Engineering, Vol. 12, Issue 2, 1990-2000. http://doi.org/10.11591/ijece.v12i2.

[48] Ofir Pele and Michael Werman. 2008. A linear time histogram metric for improved SIFT matching. In: Forsyth D., Torr P., Zisserman A. (eds) Computer Vision – ECCV 2008. ECCV 2008. Lecture Notes in Computer Science, Vol. 5304. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-88690-7_37.

[49] Ofir Pele and Michael Werman. 2009. Fast and robust earth mover's distances. IEEE 12th International Conference on Computer Vision, 460-467. 10.1109/ICCV.2009.5459199.

[50] Matt Kusner, Yu Sun, Nicholas I. Kolkin and Kilian Q. Weinberger. 2015.. From Embeddings To Document Distances. In Proceedings of the 32nd International Conference on International Conference on Machine Learning, Vol. 37 (ICML'15). JMLR.org, 957–966.

[51] Chim-Yew Lin. 2004. ROUGE: a Package for Automatic Evaluation of Summaries. Proceedings of the Workshop on Text Summarization Branches Out (WAS 2004), Barcelona, Spain, 74–81.

[52] Jovi D'Silva and Uzzal Sharma. 2021. Automatic Text Summarization of Konkani Folk Tales Using Supervised Machine Learning Algorithms and Language Independent Features. IETE Journal of Research, Taylor & Francis. https://doi.org/10.1080/03772063.2021.1987993.

[53] H. P. Luhn. 1958. The Automatic Creation of Literature Abstracts, IBM Journal of Research and Development, Vol. 2, No. 2, 159–165. http://www.research.ibm.com/journal/rd/022/luhn.pdf.

[54] Ricardo Campos, Vitor Mangaravite, Arian Pasquali, Alipio Jorge, Celia Nunes and Adam Jatowt. 2020. YAKE! Keyword Extraction from Single Documents using Multiple Local Features. In Information Sciences Journal. Elsevier, Vol. 509, 257- 289, ISSN 0020-0255. https://doi.org/10.1016/j.ins.2019.09.013.

[55] Jovi D'Silva and Uzzal Sharma. 2020. Unsupervised Automatic Text Summarization of Konkani Texts using K-means with Elbow Method. International Journal of Engineering Research and Technology, ISSN 0974-3154, Vol. 13, No. 9, 2380-2384. https://dx.doi.org/10.37624/IJERT/13.9.2020.2380-2384.

Just Accepted